

Evidence - Generated Proof

Measured runs, failure captures, the ten-case matrix and vision measurements

EdgePilot AI - Team 15

Documentation set, 26 August 2026

4 documents in this volume

evidence/ - generated proof

Work package: benchmark and DevOps, with the vision artefacts belonging to the vision benchmark.

What this folder is

Artefacts a reviewer can read **without running anything**. That is the whole design constraint. A passing test proves a behaviour to whoever runs the suite; a committed artefact proves it to whoever opens the pull request.

Every file here is written by a script, never by hand, and every one of them leads with two fields:

`what_this_proves` and `what_this_does_not_prove`. The second field is the reason the folder is worth trusting.

What happened here

`evidence/evaluation/` is new - it holds the ten-case matrix the submission checklist asks for, which previously existed only as behaviour scattered across the unit suite.

Two live vision measurements were produced, the first ones the project has ever had. They exist only because of a one-character fix: the evidence writer validated filenames as lowercase-only and then built them from an ISO timestamp containing T and Z, so the live path had never once succeeded.

The `.gitignore` rule `/evidence/vision-benchmark/live-*.json` was silently excluding those runs. One is now explicitly un-ignored with the reasoning written beside the rule.

Files

Subfolders only.

Subfolder	What is in it
<code>benchmark/</code>	Measured provider runs, failure-mode captures, the clean-start log
<code>evaluation/</code>	The ten-case evaluation matrix
<code>vision-benchmark/</code>	Controlled and live image-classification runs

Connected folders

- `scripts/` - everything here is written by something there.
- `src/app/evidence/` - the page that renders these files in the browser.

Referenced from

These folders point here. Each link below resolves in both directions.

- `/`

evidence/benchmark/ - measured provider runs

Work package: benchmark and provider layer.

What this folder is

What actually happened when this application talked to a real model, and what it did when things went wrong.

What happened here

Nothing in this folder was regenerated during the changes described in 00-PROJECT-RECORD.md, and that is worth stating plainly rather than quietly fixing: **measured-ollama-llama3.2_latest.json predates the hardware-fit and privacy work**. It still records "hardwareFit": 50 - the constant every device received - and "privacyScore": 100 from the era when privacy was averaged into readiness. Read as a historical record it is accurate. Read as a description of current behaviour it is wrong, and re-running `npm run bench:run` would replace it.

Files

File	What it is
measured-ollama-llama3.2_latest.json	A real 5-iteration Ollama run: 100% success, mean latency 2,758.9 ms, p50 1,542.1 ms, mean TTFT 1,305.3 ms, 91.3 tokens/s, 623 output tokens. Carries the full environment block - CPU, cores, memory, accelerator - so the figures can be read in context
measured-ollama-llama3.2_latest.5iter-backup.json	The prior capture of the same run, kept for comparison
failure-modes.json	Timeout, fallback and error-classification behaviour driven through the real adapters and the real runner. Records whether every scenario behaved as documented, and lists mismatches if not
clean-start.log	The clean-environment setup sequence with real commands, real exit codes and real output. Explicitly un-ignored in .gitignore, because the blanket *.log rule would otherwise have swallowed a deliverable

Connected folders

- `scripts/benchmark/` - `run-benchmark.ts`, `capture-failure-evidence.ts` and `capture-clean-start.ts` write these three.
- `src/modules/benchmark/` - the code being measured.
- `src/app/evidence/` - renders them.

Referenced from

These folders point here. Each link below resolves in both directions.

- `docs/benchmark/`
- `evidence/`

evidence/evaluation/ - the ten-case matrix

Work package: benchmark and provider layer.

What this folder is

One artefact: the reproducible ten-case evaluation the submission checklist asks for, covering normal, malformed, ambiguous, injection, missing-evidence and provider-failure behaviour.

What happened here

The folder is new. The behaviours it covers already existed in the unit suite, but a reviewer had to go and find them across several files. `npm run eval:matrix` now runs them in one place and writes a single readable artefact, and exits non-zero if any case misbehaves.

Every case drives **real module code** - the same zod schemas, the same normaliser, the same readiness calculator the application uses. No network call is made and no result is transcribed by hand.

The injection cases are worth understanding. EdgePilot never *acts* on model output, so the threat is not command execution; it is model output being **interpreted as a verdict**. The defence is exact matching against a closed label set, with anything else recorded as `invalid_output` rather than guessed at. Case 9 exists to assert a different rule: a missing hardware fit stays `null` and is never rendered as `0`.

Files

File	What it is
<code>ten-case-matrix.json</code>	10 cases in 6 categories - 1 normal, 3 malformed, 1 ambiguous, 3 injection, 1 missing-evidence, 1 provider-failure. Current state: all 10 behaved as documented, no failed case ids. Includes the injection threat model in the artefact itself

Connected folders

- `scripts/evaluation/` - the script that writes this file.
- `src/modules/benchmark/core/services/` - the readiness calculator and assessors the cases drive.
- `src/modules/vision-benchmark/core/` - the normaliser the injection cases exercise.

Referenced from

These folders point here. Each link below resolves in both directions.

- `evidence/`
- `src/app/evaluation/`
- `src/app/evidence/`
- `tests/dashboard/`
- `tests/evaluation/`

evidence/vision-benchmark/ - image classification runs

Work package: vision benchmark.

What this folder is

Four artefacts: two controlled runs against scripted providers, and two live runs against a real model on real hardware.

Each carries the full reproducibility set - schema version, workload id and version, dataset id, manifest version **and its SHA-256**, preprocessing version, prompt version, execution mode, provider, model, git commit, start and completion timestamps, every per-sample record, the metrics, the thresholds, and whether it passed. Two runs are comparable only when those versions match, which is why they are all recorded rather than assumed.

What happened here

The two live-`*` files are new, and they are the first real measurements this project has produced. Before them the live path had never worked: the evidence writer validated filenames against a lowercase-only pattern and then built them from an ISO timestamp containing T and Z, so every live run failed at the write step.

The two live runs are the same model on the same images and differ by one change - the second was taken after a warm-up classification was added, and its median latency is **529.6 ms against the first run's 1322.1 ms**. That gap is the model load, not the model.

`.gitignore` ignores live-`*`.json by default, because live runs are machine-specific and would otherwise accumulate from every developer. The second one is explicitly un-ignored, with the reason written beside the rule.

Files

File	What it is
live-ollama-llava-latest-2026-08-23t20-03-01-476z.json	The reference measurement. llava:latest, 21 samples, 14 correct: 66.7% accuracy, macro-F1 0.667, 0% invalid output, 100% request success, median 529.6 ms, p95 721.4 ms. "passed": false - it fails its own 0.80 accuracy and 0.75 macro-F1 gates. Committed <i>because</i> it fails
the earlier pre-warm-up run	Median 1322.1 ms against the kept run's 529.6 ms. Git-ignored, so it is not in this repository - the figure is quoted here because the comparison is the point, not the file
controlled-ollama.json	Deterministic run against a scripted local provider - proves the pipeline, not the model
controlled-gemini.json	The same for the cloud provider shape

Connected folders

- datasets/vision-benchmark/ - the 21 samples these measure.
- scripts/vision-benchmark/ - run.ts writes the live files, generate-controlled-evidence.ts the others.
- src/modules/vision-benchmark/infrastructure/ - evidence-store.ts, the file that had the filename bug.
- src/app/evidence/ - renders them.

Referenced from

These folders point here. Each link below resolves in both directions.

- docs/vision-benchmark/
- evidence/
- src/modules/vision-benchmark/